

COMP20008 Elements of Data Processing



Lecture 1: Semi-Structured Data Formats (“the many forms of data”)

Semester 2 2026

Subject Coordinator: **Dr Ekaterina Vylomova**

Co-lecturer: **Dr Hanzhuo (Vivian) Ma**



Announcements

1. We've got one more workshop slot:
Tue 16:00 18:00 PAR-104-L1-103-Collaborative Learning Space
2. Correction #1 : A2 groups should be **3-4 members** (ideally, 3)!
3. Correction #2: A1 Oral Assessment will be on **Week 7 (based on ESS scores; one week earlier to provide more time to incorporate feedback and prepare A2)!**

Yige Song's PhD completion Seminar



Yige Song's PhD completion Seminar



Identify
research
gap

Yige Song's PhD completion Seminar

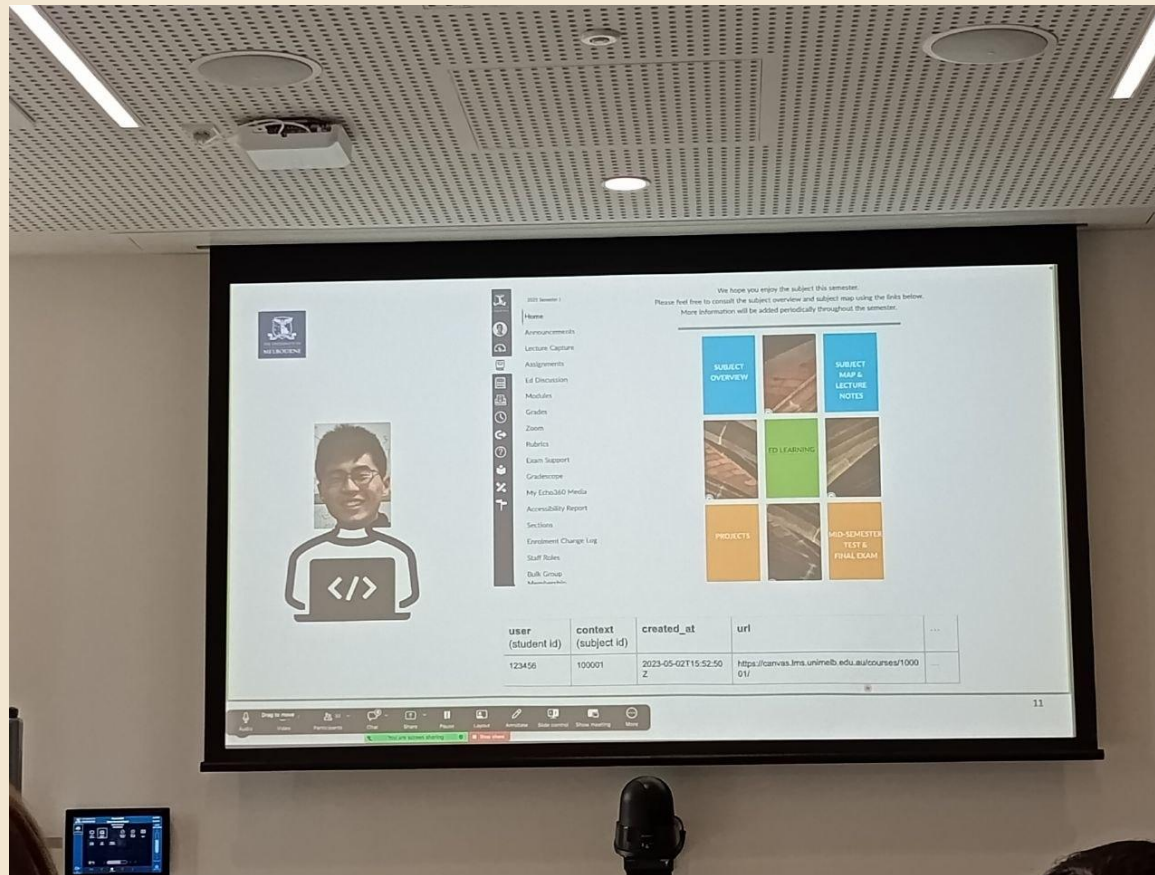


Yige Song's PhD completion Seminar

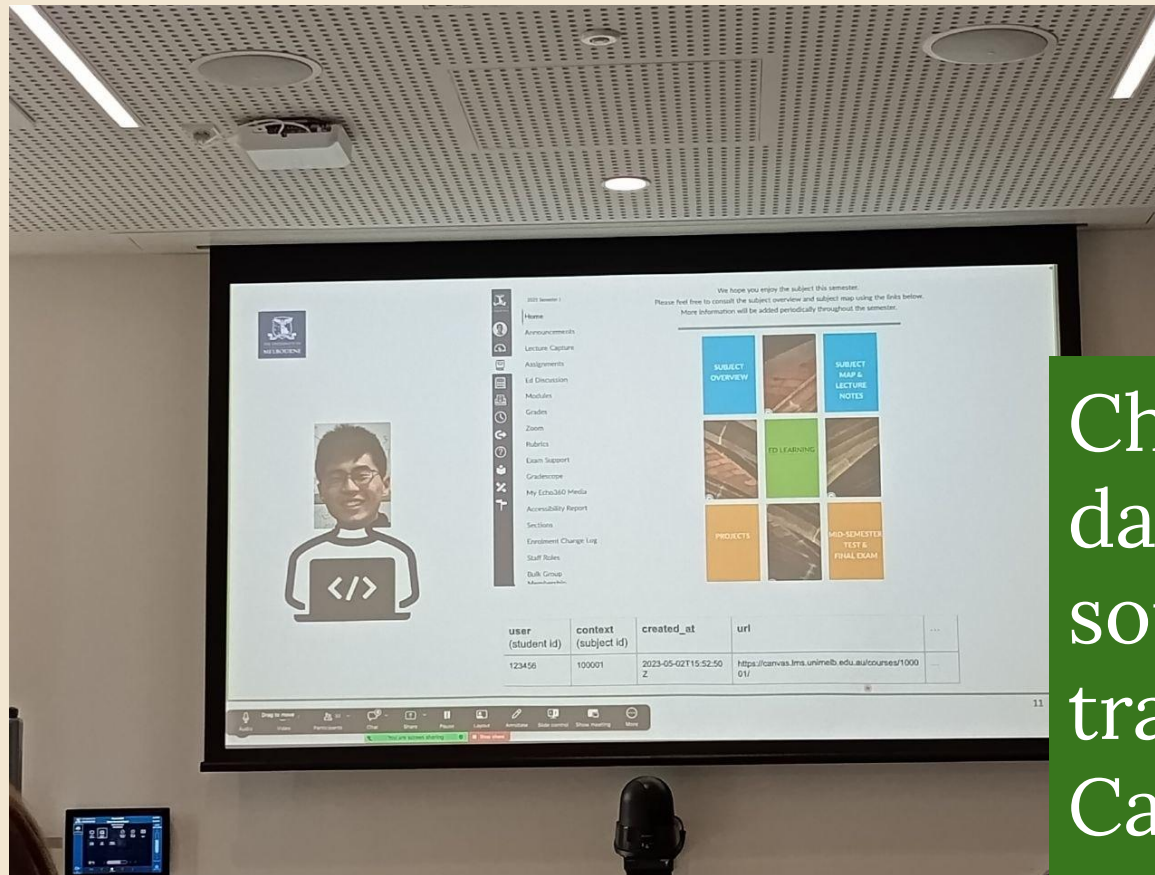


State aim
and
Research
Questions

Yige Song's PhD completion Seminar



Yige Song's PhD completion Seminar



Choose the data sources: traces on Canvas

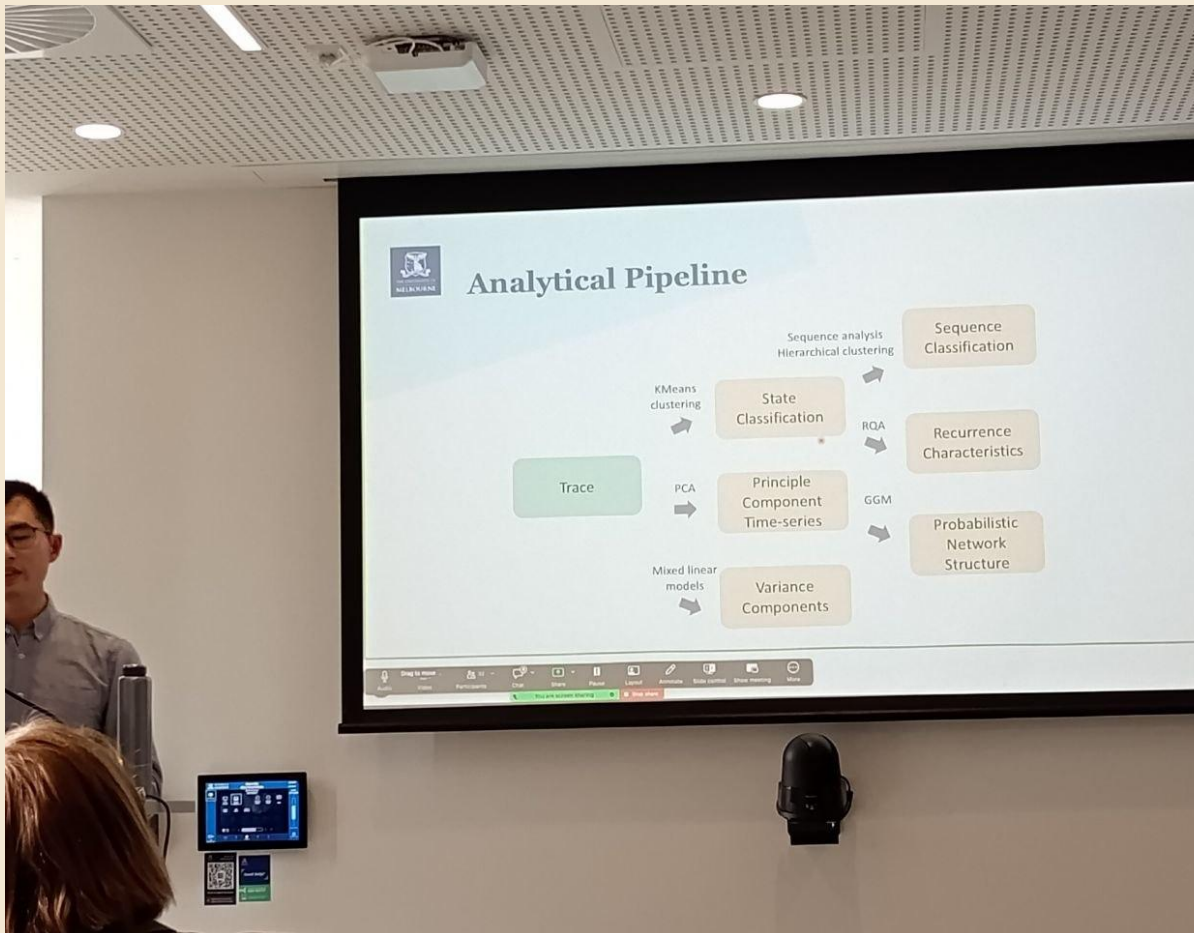
Yige Song's PhD completion Seminar



Survey students:
meta-cog
awareness; SRL;
wellbeing;
motivation

Canvas traces:
interactions;
sessions;
groupings

Yige Song's PhD completion Seminar



Do analysis
(clustering;
classification;
PCA; regression;
etc)

and **draw
conclusions**

Yige Song's PhD completion Seminar



Do analysis
(clustering;
classification;
PCA; regression;
etc)

and **draw
conclusions**

Lecture 1 Recap

Discrete or continuous?

There is no HARD boundary.

Discrete vs Continuous is:

- **Not** about the object
- **Not** about binary storage
- **About** how you **model the variable**

Final Check: What about **Age**? Discrete or Continuous?

Discrete or continuous?

There is no HARD boundary.

Discrete vs Continuous is:

- **Not** about the object
- **Not** about binary storage
- **About** how you **model the variable**

Final Check: What about **Age**? Discrete or Continuous?

In principle:

You are getting older every millisecond → **continuous**

In practice:

“I am 20 years old.” → **discrete**

Representing Numerical Data

Number systems

- binary, decimal, octal, hexadecimal
- symbols:
 - Binary (base 2): 0, 1
 - Decimal (base 10): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
 - Octal (base 8): 0, 1, 2, 3, 4, 5, 6, 7
 - Hexadecimal (base 16): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F
- The value 15 is
 - $1111_2 = 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$
 - $17_8 = 1 \times 8^1 + 7 \times 8^0$
 - $F_{16} = 15 \times 16^0$
- $4EBA_{16} = 4 \times 16^3 + 14 \times 16^2 + 11 \times 16^1 + 10 \times 16^0 = 20154$

(E)(B)(A)

Representing Non-numerical Data

For text, assign each character a number

ASCII code (outdated)

- 128 characters for English
- e.g., 011000001 is the number assigned to 'a'

Unicode

- UTF16, UTF8
 - Larger code space for different languages.
 - e.g., U+006F is the character 'o'
 - For the character '人', the Unicode is U+4EBA

Sets and sequences of data items

- Lists or sequences (vectors) of individual items
- Items of all kinds, sometimes even mixed
 - Text: list of words
 - Time series: list of (timepoint, value) pairs
 - DNA sequence: list of symbols A,T,G,C

Week 1 Workshop & Lecture 1 Alignment

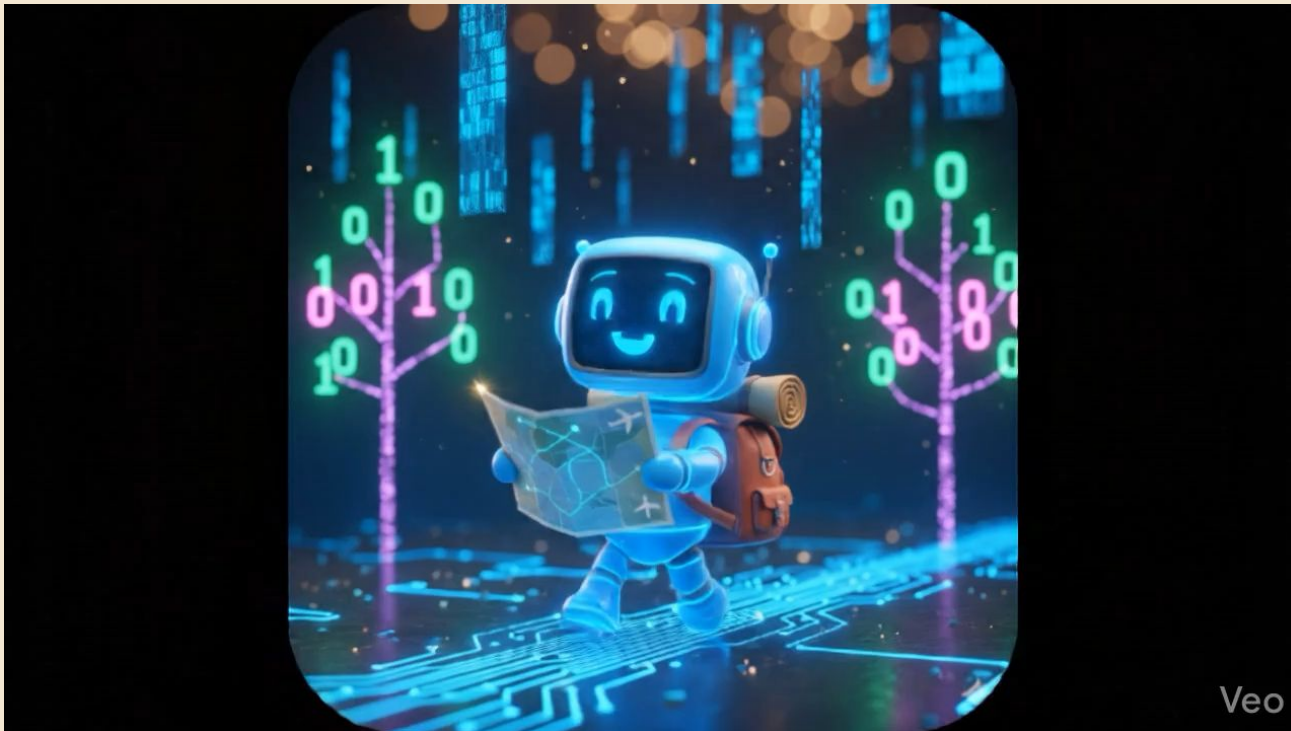
Week 1 Workshop Goal

- Load structured data (CSV) using Pandas
- Manipulate tabular data (filter, sort, group, aggregate)
- Compute summary statistics (e.g., occupancy rate)
- Understand Series and DataFrame structures

Related Lecture 1 Slides (Part 2)

- The Core of Data Processing
Workshop practices *Data Input* and *Data Cleanup & Manipulation* in the pipeline.
- Data Types (Numerical vs Non-numerical)
Workshop applies these concepts to real columns (e.g., `tickets_sold` = numerical, `classification` = categorical).
- Discrete vs Continuous (Modelling)
Workshop models counts and rates (e.g., occupancy rate) highlighting that discrete vs continuous is about modelling.

Data Travels Between Systems



Data Travels Between Systems

**COFFEE
PURCHASE**



Data Travels Between Systems

**COFFEE
PURCHASE**



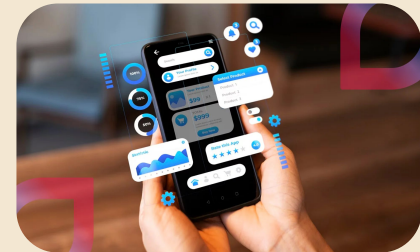
POS SYSTEM



Accounting



MOBILE



Data Travels Between Systems

COFFEE
PURCHASE



POS SYSTEM

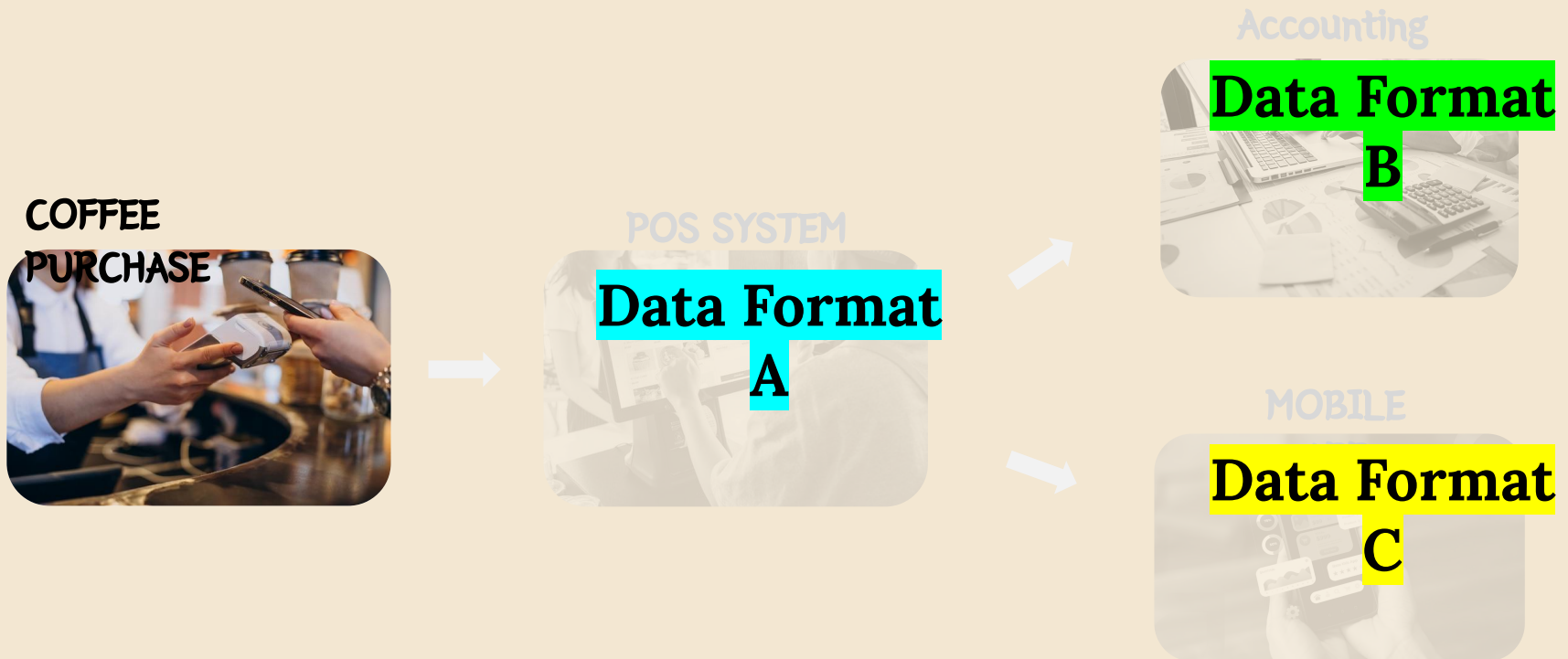


How does data move
between these systems?

Accounting
System



Data Travels Between Systems



Semi-structured Data Formats

Format	Purpose	Structure	Semantics	Typical Use
Simple Lists: CSV/TSV	Tabular data	Flat	External	Data analysis
Styling the Web: HTML	Display	Document hierarchy	Weak	Web pages
Extended Markup Language: XML	Data storage, interpretable labels	Hierarchical	Strong	Config, standards
Programming Data: JSON	Data exchange	Hierarchical	Moderate	APIs

Semi-structured Data Formats

Format	Purpose	Structure	Semantics	Typical Use
Simple Lists: CSV/TSV	Tabular data	Flat	External	Data analysis
Styling the Web: HTML	Display	Document hierarchy	Weak	Web pages
Extended Markup Language: XML	Data storage, interpretable labels	Hierarchical	Strong	Config, standards
Programming Data: JSON	Data exchange	Hierarchical	Moderate	APIs

CSV: Comma Separated Values

A1		fx		Date	
	A	B	C	D	E
1	Date	17/02/2014	22/02/2014	28/02/2014	
2	BP	130/80	140/90	131/87	
3	Glucose	4.5	6	5.3	
4	Heart rate	55	70	57	
5					
6					
7					
8					

**This looks familiar, right?
A simple table in a
spreadsheet!**

CSV: Comma Separated Values

	A1				
	A	B	C	D	E
1	Date	17/02/2014	22/02/2014	28/02/2014	
2	BP	130/80	140/90	131/87	
3	Glucose	4.5	6	5.3	
4	Heart rate	55	70	57	
5					
6					
7					
8					

**But
Underneath...**

text_example.csv - Notepad

File Edit Format View Help

```
Date,17/02/2014,22/02/2014,28/02/2014
BP,130/80,140/90,131/87
Glucose,4.5,6,5.3
Heart rate,55,70,57
```

How does the
computer
understand this?

CSV: Comma Separated Values

The Simplest Data Format

- Stores data in rows and columns

**But
Underneath...**

How does the
computer
understand this?

	A1				
	A	B	C	D	E
1	Date	17/02/2014	22/02/2014	28/02/2014	
2	BP	130/80	140/90	131/87	
3	Glucose	4.5	6	5.3	
4	Heart rate	55	70	57	
5					
6					
7					
8					

text_example.csv - Notepad

File Edit Format View Help

```
Date,17/02/2014,22/02/2014,28/02/2014
BP,130/80,140/90,131/87
Glucose,4.5,6,5.3
Heart rate,55,70,57
```

CSV = values separated by commas

CSV: Comma Separated Values

The Simplest Data Format

- Stores data in rows and columns

**But
Underneath...**

How does the
computer
understand this?

	A1				
	A	B	C	D	E
1	Date	17/02/2014	22/02/2014	28/02/2014	
2	BP	130/80	140/90	131/87	
3	Glucose	4.5	6	5.3	
4	Heart rate	55	70	57	
5					
6					
7					
8					

```
text_example.csv - Notepad
File Edit Format View Help
Date,17/02/2014,22/02/2014,28/02/2014
BP,130/80,140/90,131/87
Glucose,4.5,6,5.3
Heart rate,55,70,57
```

CSV = values separated by **commas**

What CSV does well

- ✓ Simple
- ✓ Human Readable (Plain text format)
- ✓ Easy to analyse
- ✓ Works with Excel / Python/ R

What CSV cannot do

- x No formatting
- x No hierarchy
- x Very weak semantics

TSV: Tab Separated Values

Same as CSV, but uses tabs instead of commas

```
Date,17/02/2014,22/02/2014,28/02/2014  
BP,130/80,140/90,131/87  
Glucose,4.5,6,5.3  
Heart rate,55,70,57
```

CSV = values separated by **commas**

```
Date 17/02/2014 22/02/2014  
28/02/2014  
BP 130/80 140/90 131/87  
Glucose 4.5 6 5.3  
Heart rate 55 70 57
```

TSV = values separated by **tabs**

CSV and TSV store exactly the same data!

They just use **different separators**
(sometimes it's better to use tabs as separators to avoid confusion)

Semi-structured Data Formats

Format	Purpose	Structure	Semantics	Typical Use
Simple Lists: CSV/TSV 	Tabular data	Flat	External	Data analysis
Styling the Web: HTML 	Display	Document hierarchy	Weak	Web pages
Extended Markup Language: XML	Data storage, interpretable labels	Hierarchical	Strong	Config, standards
Programming Data: JSON	Data exchange	Hierarchical	Moderate	APIs

HTML: HyperText Markup Language

Computers originally showed plain text only

```
Welcome to COMP20008!  
Workshops start on Week 1!  
Announcement  
Contact
```

But humans like structure and *emphasis*

HTML: HyperText Markup Language

Computers originally showed plain text only

Welcome to COMP20008!

Make this big

Workshops start on Week 1!

Make this bold

Announcement

Contact

Make this as a list

Recall
COMP10001!

HTML: HyperText Markup Language

Computers originally showed plain text only

```
<h1>Welcome to COMP20008! </h1>  
<b>Workshops start on Week 1!</b>  
<ul>  
<li>Announcement</li>  
<li>Contact</li>  
</ul>
```


HTML: HyperText Markup Language

Computers originally showed plain text only

```
<h1>Welcome to COMP20008! </h1>
<b>Workshops start on Week 1!</b>
<ul>
<li>Announcement</li>
<li>Contact</li>
</ul>
```

Welcome to COMP20008!

Workshops start on Week 1!

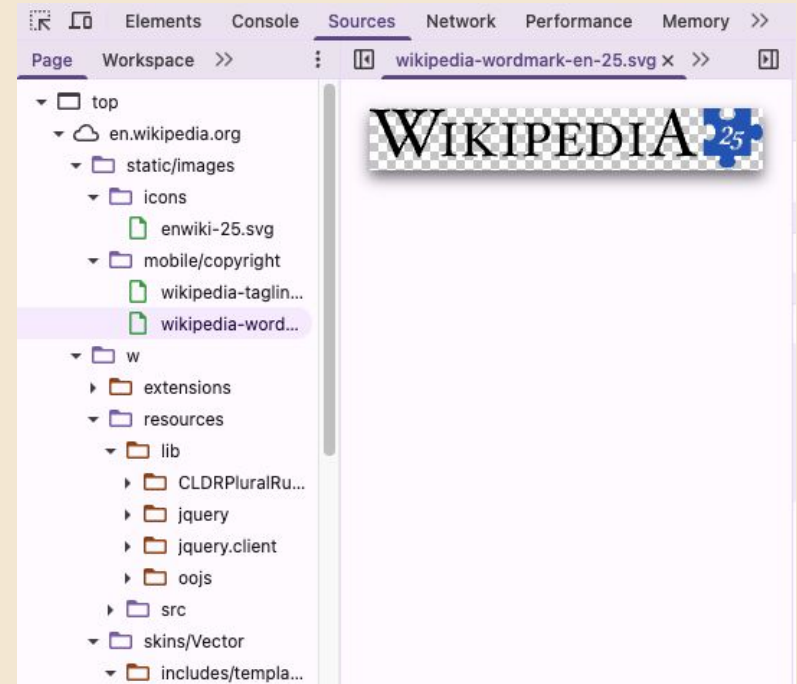
- Announcement
- Contact

Something has to tell the computer to make this large!

36

Inside of an HTML page!

- **Many files**, one for each picture, one for each icon, etc., and one loooooong one for the main text
- The main file links them all together

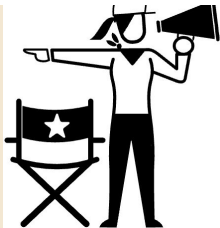




Old Quad, the original building of the University of Melbourne



The original University of Melbourne building, 1857, Victoria Illustrated collection, State Library Victoria.



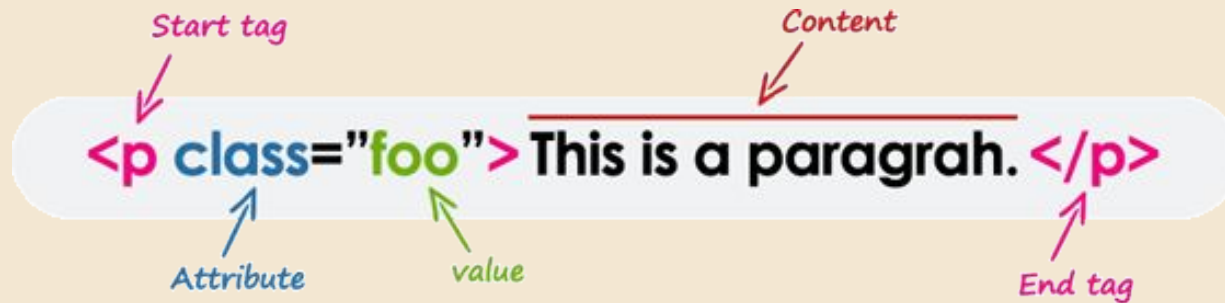
HTML
=Director of a Scene

```
<meta property="mw:PageProp/toc">
▶ <div class="mw-heading mw-heading2">⋮ </div>
▼ <figure class="mw-default-size mw-halign-right" typeof="mw:File/Thumb">
▶ <a href="/wiki/File:Melbourne University grand building.jpg" class="mw-file-description">⋮ </a>
▼ <figcaption>
::before
"Old Quad, the original building of the University of Melbourne" = $0
</figcaption>
</figure>
▶ <div class="mw-heading mw-heading3">⋮ </div>
▼ <figure class="mw-default-size" typeof="mw:File/Thumb">
▼ <a href="/wiki/File:The University of Melbourne, 1857.jpg" class="mw-file-description">

::after
</a>
▼ <figcaption>
::before
"The original University of Melbourne building, 1857, Victoria Illustrated collection, State Library
Victoria."
</figcaption>
</figure>
▶ <figure class="mw-default-size" typeof="mw:File/Thumb">⋮ </figure>
▶ <p>⋮ </p>
▶ <p>⋮ </p>
▶ <p>⋮ </p>
▶ <figure class="mw-default-size" typeof="mw:File/Thumb">⋮ </figure>
▶ <div class="mw-heading mw-heading3">⋮ </div>
```

The main one has HTML formatting and positioning commands all over !

HTML: Hypertext Markup Language



- Marked up with **elements**, delineated by start and end **tags** (e.g., a paragraph, heading or itemised list)
- **Tags**: keywords contained in pairs of angle brackets
 - Tags are **not case sensitive**
 - Elements can have attributes. Ordering of attributes is not significant: `<h1 class="city" id="c2">`
 - Not all elements need both start and end tags: `<br/>`

Try yourself:

https://www.w3schools.com/html/tryit.asp?filename=tryhtml_default

Example: Subject guide

Data

- Year of offer = 2019
- Subject level = Undergraduate level 2
- Subject code = COMP20008
- Campus = Parkville
- Availability = Semester 1, Semester 2

Can you make the browser to show it like this?

Year of offer	2019
Subject level	Undergraduate level 2
Subject code	COMP20008
Campus	Parkville
Availability	Semester 1, Semester 2

Example: Subject guide in HTML

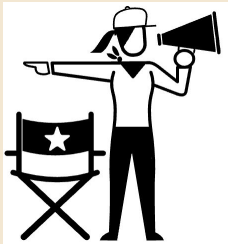
start a table
table row

end table row

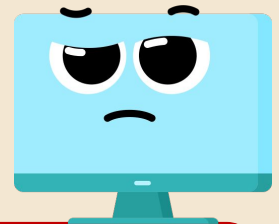
```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>COMP20008 Subject Infomation</title>
6 </head>
7 <body>
8   <table border="1">
9     <tr>
10      <th>Year of offer</th>    Row header
11      <td>2019</td>    Row filler
12    </tr>
13    <tr>
14      <th>Subject level</th>
15      <td>Undergraduate level 2</td>
16    </tr>
17    <tr>
18      <th>Subject code</th>
19      <td>COMP20008</td>
20    </tr>
21    <tr>
22      <th>Campus</th>
23      <td>Parkville</td>
24    </tr>
25    <tr>
26      <th>Availability</th>
27      <td>Semester 1, Semester 2</td>
28    </tr>
29  </table>
30 </body>
31 </html>
```


Generalizing HTML

HTML is great for telling the browser
how things should look



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>COMP20008 Subject Information</title>
6 </head>
7 <body>
8   <table border="1">
9     <tr>
10      <th>Year of offer</th>
11      <td>2019</td>
12    </tr>
13    <tr>
14      <th>Subject level</th>
15      <td>Undergraduate level 2</td>
16    </tr>
17    <tr>
18      <th>Subject code</th>
19      <td>COMP20008</td>
20    </tr>
21    <tr>
22      <th>Campus</th>
23      <td>Parkville</td>
24    </tr>
25    <tr>
26      <th>Availability</th>
27      <td>Semester 1, Semester 2</td>
28    </tr>
29  </table>
30 </body>
31 </html>
```



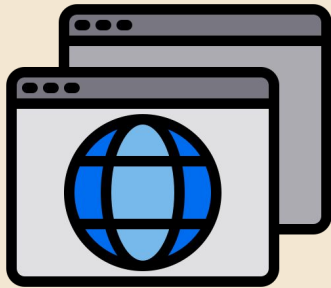
BUT Can we specify what's inside the tags (add more semantics)?

Appearance VS Meaning

Assume that you would like to announce “**Exam on Friday**”

HTML

```
<b>Exam on Friday</b>
```



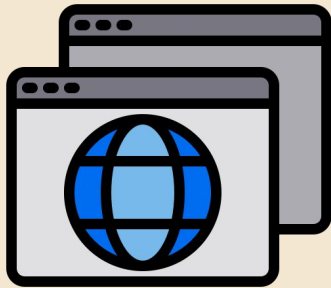
Browser reads
“make the text bold”

Appearance VS Meaning

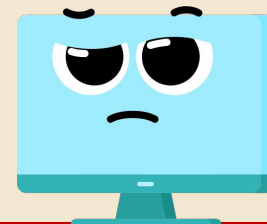
Assume that you would like to announce “**Exam on Friday**”

HTML

```
<b>Exam on Friday</b>
```



Browser reads
“make the text bold”



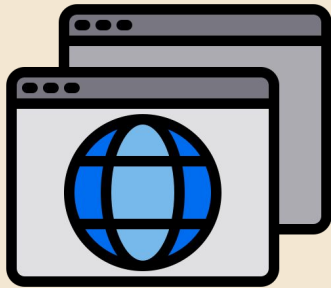
It is just a TEXT

Appearance VS Meaning

Assume that you would like to announce “**Exam on Friday**”

HTML

```
<b>Exam on Friday</b>
```



Browser reads
“make the text bold”

What if...?

```
<announcement>  
Exam on Friday  
</announcement>
```



OK! This is an announcement

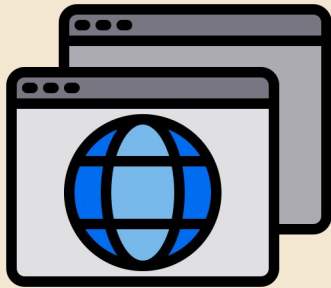
Appearance VS Meaning

Assume that you would like to announce “**Exam on Friday**”

HTML

General tags

```
<b>Exam on Friday</b>
```



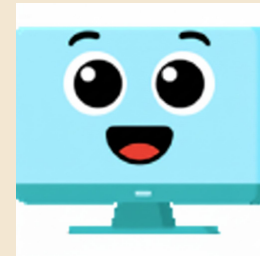
Tells the browser
HOW to present it

Presentation

XML

Custom tags

```
<announcement>  
Exam on Friday  
</announcement>
```



Tells the computer
WHAT it is inside

Content

Semi-structured Data Formats

Format	Purpose	Structure	Semantics	Typical Use
Simple Lists: CSV/TSV 	Tabular data	Flat	External	Data analysis
Styling the Web: HTML 	Display	Document hierarchy	Weak	Web pages
Extended Markup Language: XML 	Data storage, interpretable labels	Hierarchical	Strong	Config, standards
Programming Data: JSON	Data exchange	Hierarchical	Moderate	APIs

XML: eXtensible Markup Language

A 'meta' mark-up language

- **Language for building languages** (e.g. SVG, MathML, RSS)
- **Extensible: user-defined tags**

```
<announcement> Exam on Friday </announcement>
```

- Separates style and content
 - Focus on **data**, not appearance
- Device and system-independent
 - XML files are plain text and work **across different systems**. (Windows, Mac, Mobile, Web, etc.)
- XML was designed to allow systems to exchange structured data (e.g. banking systems, enterprise software)

```
<profile>  
  <name>David</name>  
  <dateofbirth>2000.04.01</dateofbirth>  
</profile>
```


XML example

```
<?xml version="1.0"?>
<customers>
  <customer>
    <firstname>David</firstname>
    <lastname>Lynn</lastname>
    <age>30</age>
    <address>
      <staddress>211 Fox Street</staddress>
      <city>Greenville</city>
    </address>
    <email>dlynn@nhs.net</email>
  </customer>
</customers>
```

Group data together

Cities in HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My Cities</title>
</head>
<body>
<section class="section">
  <h1>Cities</h1>
  <div class="container">
    <div id="Results">
      <h2 class="city">Paris</h2>
      <p>Paris is the capital of France.</p>
      <h2 class="city">Tokyo</h2>
      <p>Tokyo is the capital of Japan.</p>
      <h2 class="city">Melbourne</h2>
      <p>Melbourne is in Victoria, Australia.</p>
    </div>
  </div>
</section>
</body>
</html>
```

Cities in XML

```
<?xml version="1.0"?>
```

Group data together

```
<cities>  
  <city>Paris</city>  
  <comment>Paris is the capital of France.</comment>  
  <city>Tokyo</city>  
  <comment>Tokyo is the capital of Japan.</comment>  
  <city>Melbourne</city>  
  <comment>Melbourne is in Victoria, Australia.</comment>  
</cities>
```

XML Syntax

- xml files should begin with declaration
 - `<?xml version="1.0"?>`
- xml files must have one single root element
 - `<cities>...</cities>`
- elements are **built with tags**, must be properly closed
 - opening `<city>` and closing `</city>`
 - empty `<br />`
- an element may have one or more attributes, attributes must be in quotes
 - `<person title="Sir">Richard</person>`
 - `<person title="Mr" age="30">James</person>`

XML Syntax

- Unlike HTML, XML code is case sensitive
 - <title> is not the same as <Title>
- Like in HTML, elements must be appropriately nested
 - <author><firstname>James</firstname></author>
 - <author><firstname>James</author> </firstname>
 - Wrong...
- comments
 - <!-- comments do not affect the document,
it's not part of the data that you want to represent -->

XML Syntax

- some characters have special meaning
 - ‘<’ and ‘&’ are strictly illegal inside an element
 - `<text>all books & videos are now < AUD 10</text>`
 - Wrong...
 - `<text>all books & videos are now < AUD 10</text>`
- **CDATA** (character data) section may be used inside XML element to include large blocks of text, which may contain special characters such as &, >
 - `<![CDATA [ ... ... ] ]>`
 - `<![CDATA [all books & videos are now < AUD 10] ]>`

XML Syntax (Example)

```
<?xml version="1.0"?>
```

Declaration

```
<catalog> One root element
```

```
  <book isbn="1-23456-789-0"> Attributes in quotes
```

```
    <title>Beyond the Clouds</title>
```

```
    <author>
```

```
      <firstname>Rebecca</firstname>
```

```
      <surname>Skye</surname>
```

```
      <picture source="rebecca.jpg" />
```

```
    </author>
```

```
  </book>
```

```
  <book isbn="0-98765-432-1">
```

```
    <title>The Final Straw</title>
```

```
    <author>
```

```
      <firstname>James</firstname>
```

```
      <surname>Last</surname>
```

```
      <picture source="james.jpg" />
```

Empty tag / self-closing tag

```
    </author>
```

```
  </book>
```

```
</catalog>
```

Exercise

Given the following data: *Yellow Balloons* cost \$9.99. What are possible XML encodings of the data?

```
<balloon>
  <colour>Yellow</colour>
  <price>9.99</price>
</balloon>
```

```
<balloon colour="Yellow">
  <price>9.99</price>
</balloon>
```

```
<balloon colour="Yellow"
price="9.99"></balloon>
```

They are all good!
There's no '**right**' answer.

It depends on what you
want to do with the info:
*what is easiest to find and use
for you and your system?*

Break – AI Meme TIME

XML: MAKING COMPUTERS WORK HARD SINCE 1998

```
1 <?xml version="1.0"
2 encoding="UTF-8"?>
3 <note>
4   <to>Developer</to>
5   <from>XML</from>
6   <message>
7     Hello. I heard you
8     like angle brackets.
9   </message>
10 </note>
```

Aww, so structured. So polite.

WHAT HUMANS SEE:

```
<data>
  <responses>
    <status>
      <code>200</code>
      <message>OK</message>
    </status>
    <result>
      <items>
        <item>
          <id>123</id>
          <name>Widget</name>
          <price>19.99</price>
        </item>
      </items>
    </result>
  </responses>
</data>
```

WHY SO MANY BRACKETS?! WHAT IS THIS, A NESTING DOLL FACTORY?!

WHAT XML REALLY IS:

We protect your data with love and layers. ❤️

I ❤️ TAGS

XML'S SUPERPOWERS:

- 🔒 Self-describing
- 🛡️ Platform independent
- 📝 Extensible
- ❤️ Still somehow not dead

UGLY? MAYBE. USELESS? NEVER.

XML: IT'S NOT PRETTY, BUT IT GETS THE JOB DONE! 🤓

XML IS TOO HEAVY...

lots of tags

long messages for APIs

COFFEE
PURCHASE



```
<transaction>
<transaction_id>839201</transaction_id>
  <product>Latte</product>
  <price currency="AUD">5.50</price>
  <payment_method>card</payment_method>
</transaction>
```

heavy to parse

verbose

...

XML IS TOO HEAVY...



lots of tags

long messages for APIs

```
<transaction>
<transaction_id>839201</transaction_id>
  <product>Latte</product>
  <price currency="AUD">5.50</price>
  <payment_method>card</payment_method>
</transaction>
```

heavy to parse

We need some lighter data format

Occam's razor: Entities must not be multiplied beyond necessity (*btw, it's also relevant to overfitting*)

Semi-structured Data Formats

Format	Purpose	Structure	Semantics	Typical Use
Simple Lists: CSV/TSV 	Tabular data	Flat	External	Data analysis
Styling the Web: HTML 	Display	Document hierarchy	Weak	Web pages
Extended Markup Language: XML 	Data storage, interpretable labels	Hierarchical	Strong	Config, standards
Programming Data: JSON	Data exchange	Hierarchical	Moderate	APIs

XML VS JSON

COFFEE

PURCHASE



XML

```
<transaction>
<transaction_id>839201</transaction_id>
  <product>Latte</product>
  <price currency="AUD">5.50</price>
  <payment_method>card</payment_method>
</transaction>
```

JSON

```
{
  "transaction_id": 839201,
  "product": "latte",
  "price": 5.50,
  "quantity": 1,
  "payment_method": "card"
}
```

...

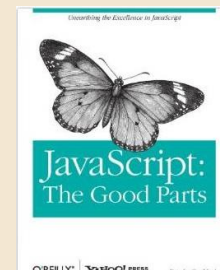
JSON: JavaScript Object Notation

Another popular and useful notation is JSON

- Representation of objects as variables to manipulate **inside your program**

Details: www.json.org

- Douglas Crockford
(pretty much alone; c.f. the development of XML by committee)
- “*Javascript: the good parts*”. O’Reilly, Yahoo Press



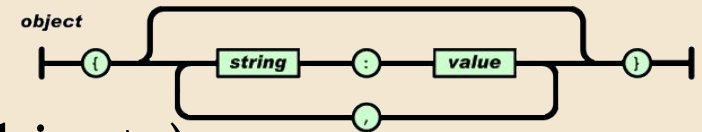
JSON object example

```
{  
  "firstName": "David",  
  "lastName": "Lynn",  
  "age": 30,  
  "address":  
    {  
      "streetAddress": "211 Fox Street",  
      "city": "Greenville",  
    },  
  "email": "dlynn@nhs.net"  
}
```

JSON syntax rules

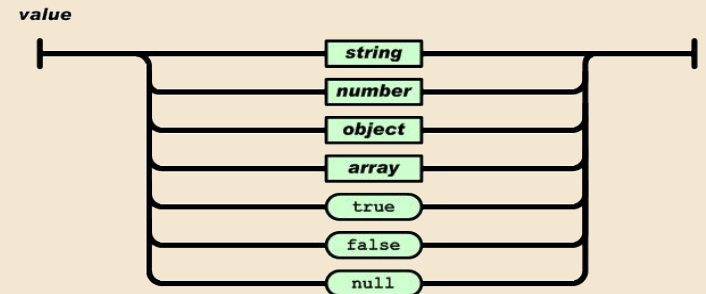
- **JSON Object**

- ◇ key – value pairs
- ◇ key is a string in **double quotes**



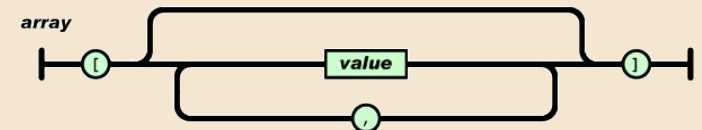
- **JSON Value** (the **types** of JSON objects):

- ◇ A number (integer or floating point)
- ◇ A string (in **double quotes**)
- ◇ A boolean (**true** or **false**)
- ◇ An array (in square brackets)
- ◇ An object (in curly braces)
- ◇ Null



- **JSON Array**

- ◇ Values in square brackets



In terms of syntax that's pretty much it!

JSON syntax examples

- JSON Object: use **{** and **}** and **,**
`{"firstName": "John", "lastName": "Doe"}`
- JSON Array: use **[** and **]** and **,**
`[
 {"firstName": "John", "lastName": "Doe"},
 {"firstName": "Anna", "lastName": "Smith"},
 {"firstName": "Peter", "lastName": "Jones"}
]`
- These objects repeat recursively down a hierarchy as needed

JSON schemas

- A *schema* is a template for data that will be created and filled in by the computer program
- You can write/define a schema using JSON itself!
- Describes the structure of other data
- Easy to validate a JSON document against its schema using a schema validator (<http://jsonschema.lint.com/draft4/>)

JSON

```
{  DATA
  "name": "Alice",
  "age": 25
}
```



JSON SCHEMAS

```
{
  "type": "object",
  "properties": {
    "name": {"type": "string"},
    "age": {"type": "number"}
  },
  "required": ["name", "age"]
}
```

Example: JSON specific instances

JSON

```
{ " CATALOG ": [
  { "CD": { "TITLE": "Empire Burlesque",
            "ARTIST": "Bon Dylan",
            "COUNTRY": "USA",
            "PRICE": { "CURRENCY": "USD",
                      "VALUE": 10.90
                    },
            "YEAR": 1985}
  },
  { "CD": { "TITLE": "Hide your heart",
            "ARTIST": "Bonnie Taylor",
            "COUNTRY": "UK",
            "PRICE": { "CURRENCY": "USD",
                      "VALUE": 9.9
                    },
            "YEAR": 1988
          }
  }
] }
```



XML

```
<CATALOG>
  <CD>
    <TITLE>Empire Burlesque</TITLE>
    <ARTIST>Bob Dylan</ARTIST>
    <COUNTRY>USA</COUNTRY>
    <COMPANY>Columbia</COMPANY>
    <PRICE CURRENCY="USD">
      10.90</PRICE>
    <YEAR>1985</YEAR>
  </CD>
  <CD>
    <TITLE>Hide your heart</TITLE>
    <ARTIST>Bonnie Tyler</ARTIST>
    <COUNTRY>UK</COUNTRY>
    <COMPANY>CBS Records</COMPANY>
    <PRICE CURRENCY="USD">9.90</PRICE>
    <YEAR>1988</YEAR>
  </CD>
</CATALOG>
```



Exercise

Represent the following information in JSON

```
<Person>
  <FirstName>Homer</FirstName>
  <LastName>Simpson</LastName>
  <Relatives>
    <Relative>Grandpa</Relative>
    <Relative>Marge</Relative>
    <Relative>Lisa</Relative>
    <Relative>Bart</Relative>
  </Relatives>
  <FavouriteBeer>Duff</FavouriteBeer>
</Person>
```

Check it is well formed
– <http://jsonlint.com> or
<https://jsonformatter.curiousconcept.com/>

Using JSON in Python



Let's recall:

What's serialisation?

What's deserialisation?

Using JSON in Python

loads: Load a JSON-formatted string into a **Python object**

dumps: Converts a **Python object** into a JSON-formatted string

```
in_str = """ {
    "Person": {
        "FirstName": "Homer",
        "LastName": "Simpson",
        "Relatives": [
            "Grandpa",
            "Marge",
            "Lisa",
            "Bart"
        ],
        "FavouriteBeer": "Duff"
    }
} """
```

```
>>> import json
>>> x = json.loads(in_str)
>>> x
{'Person': {'FirstName': 'Homer', 'LastName': 'Simpson', 'Relatives': ['Grandpa', 'Marge', 'Lisa', 'Bart'], 'FavouriteBeer': 'Duff'}}
>>> type(x)
<class 'dict'>
>>>
>>> json_str = json.dumps(x)
>>> print(json_str)
{"Person": {"FirstName": "Homer", "LastName": "Simpson", "Relatives": ["Grandpa", "Marge", "Lisa", "Bart"], "FavouriteBeer": "Duff"}}
>>> type(json_str)
<class 'str'>
>>>
```

Using JSON in Python

loads: Load a JSON-formatted string into a **Python object**

dumps: Converts a **Python object** into a JSON-formatted string

```
in_str = """ {
  "Person": {
    "FirstName": "Homer",
    "LastName": "Simpson",
    "Relatives": [
      "Grandpa",
      "Marge",
      "Lisa",
      "Bart"
    ],
    "FavouriteBeer": "Duff"
  }
}
```

recall: **load()** and **dump()**
will load from and save
to a file

```
>>> import json
>>> x = json.loads(in_str)
>>> x
{'Person': {'FirstName': 'Homer', 'LastName': 'Simpson', 'Relatives': ['Grandpa', 'Marge', 'Lisa', 'Bart'], 'FavouriteBeer': 'Duff'}}
>>> type(x)
<class 'dict'>
>>>
>>> json_str = json.dumps(x)
>>> print(json_str)
{"Person": {"FirstName": "Homer", "LastName": "Simpson", "Relatives": ["Grandpa", "Marge", "Lisa", "Bart"], "FavouriteBeer": "Duff"}}
>>> type(json_str)
<class 'str'>
>>>
```

Using JSON in Python

loads: Load a JSON-formatted string into a **Python object**

dumps: Converts a **Python object** into a JSON-formatted string

Recall: what happens to lists vs tuples?

Using JSON in Python

loads: Load a JSON-formatted string into a **Python object**

dumps: Converts a **Python object** into a JSON-formatted string

Recall: what happens to lists vs tuples?

```
>>> my_tuple = (3, "35", True, 4)
>>> my_tuple_str = json.dumps(my_tuple)
>>> print(my_tuple_str)
[3, "35", true, 4]
>>> x = json.loads(my_tuple_str)
>>> print(x)
[3, '35', True, 4]
>>>
```

Sets are not
serialisable!

```
>>> my_set = {"a", "d", "b"}
>>> print(my_set)
{'d', 'b', 'a'}
>>> my_set_str = json.dumps(my_set)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/lib/python3.10/json/__init__.py", line 231, in dumps
    return _default_encoder.encode(obj)
  File "/usr/lib/python3.10/json/encoder.py", line 199, in encode
    chunks = self.iterencode(o, _one_shot=True)
  File "/usr/lib/python3.10/json/encoder.py", line 257, in iterencode
    return _iterencode(o, 0)
  File "/usr/lib/python3.10/json/encoder.py", line 179, in default
    raise TypeError(f'Object of type {o.__class__.__name__} '
TypeError: Object of type set is not JSON serializable
>>>
```

JSON summary

- JavaScript Object Notation
- Lightweight, streamlined, standard method of data exchange
- Originally designed to speed up client/server interactions by running in the client browser
- Native JavaScript, so it can be executed as code
- Be used to represent any kind of semi-structured data
- Lacks context and schema definitions

HTML vs XML vs JSON

- HTML is **not extensible** and is concerned with **formatting, not semantics** (meaning)
- XML allows complex schema definitions
 - Extensible**, elements represent **semantics**
 - makes you consider the data design more closely
- JSON is more **lightweight and compact** and easy to parse
 - Appeals to programmers looking for speed and efficiency

Semi-structured Data Formats

Format	Purpose	Structure	Semantics	Typical Use
Simple Lists: CSV/TSV 	Tabular data	Flat	External	Data analysis
Styling the Web: HTML 	Display (form)	Document hierarchy	Weak	Web pages
Extended Markup Language: XML 	Data storage, interpretable labels	Hierarchical	Strong	Config, standards
Programming Data: JSON	Data exchange	Hierarchical	Moderate	APIs

Week 2 Workshop & Lecture 1 Alignment

- Will provide practical experience with XML and JSON
- Processing CSV, XML, and JSON data
- Manipulate pandas data frames and basic python data structures such as list and dictionary

END – AI Meme TIME

JSON : THE COOL, EASYGOING ONE

```
1 {  
2   "name": "Doggo",  
3   "type": "Good Boy",  
4   "skills": [  
5     "Fetch",  
6     "Napping",  
7     "Stealing Socks"  
8   ],  
9   "bork": true  
10 }
```



XML : OVERCOMPLICATED SINCE 1998

```
1 <?xml version="1.0" encoding="UTF-8"?>  
2 <dog>  
3   <name>Doggo</name>  
4   <type>Good Boy</type>  
5   <skills>  
6     <skill>Fetch</skill>  
7     <skill>Napping</skill>  
8     <skill>Stealing Socks</skill>  
9   </skills>  
10  <bork>true</bork>  
11 </dog>  
12  
13 <!-- Why so many brackets?! -->  
14 <!-- Is this data or ☹️ summoning ritual? -->  
15  
16
```



JSON: BECAUSE NOBODY HAS TIME FOR EXTRA ANGLE BRACKETS.